다시 반갑습니다.

Agent Framework 세미나 · 2차

실행 구조와 개발 책임



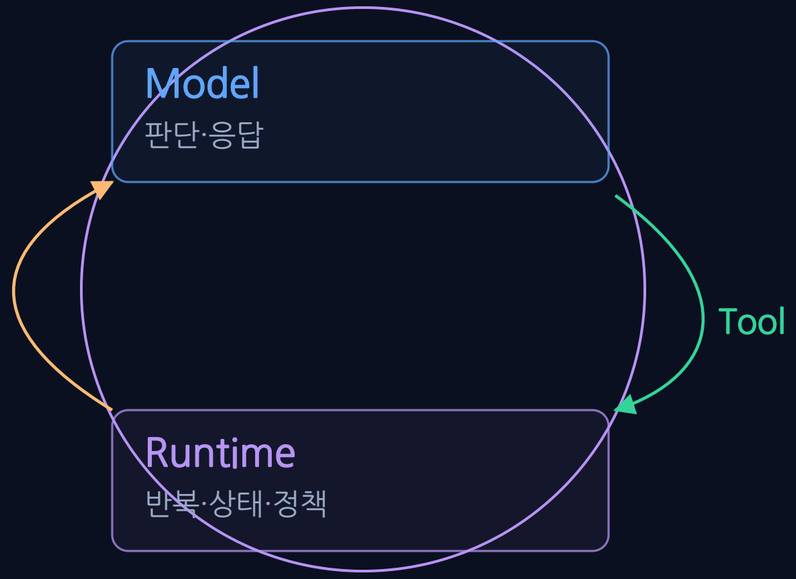
이제 실제 구현을 들여다봅니다

개념의 합의에서, 실제 동작과 책임의 경계로

AGENT FRAMEWORK

Inside the Implementations

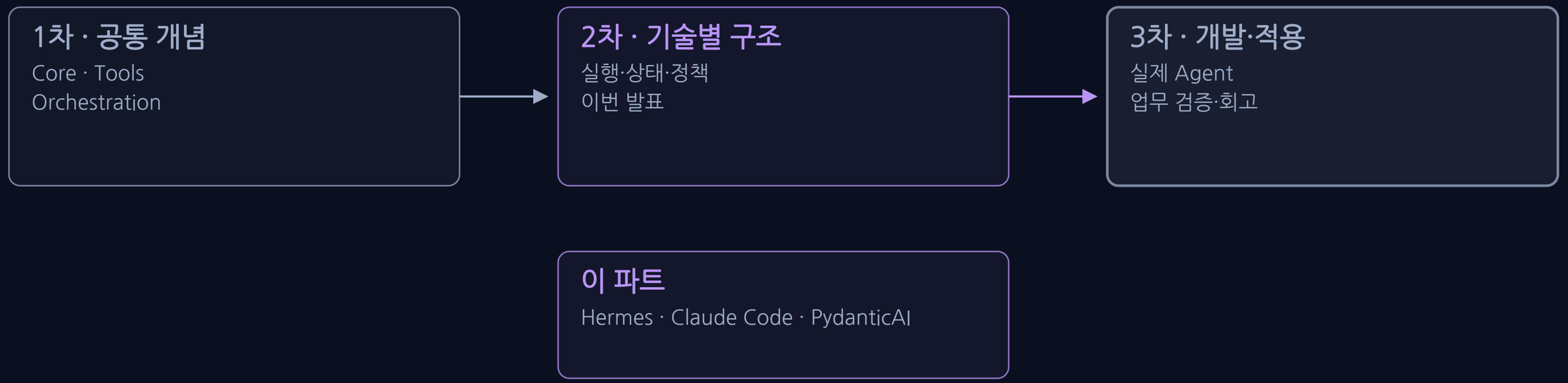
같은 구성요소, 서로 다른 실행 책임



무엇을 제공받고, 무엇을 직접 만드는가?

2차에서 맡는 범위

기초는 전제로 두고, 기술별 실행 구조를 살펴봅니다.



다른 발표자

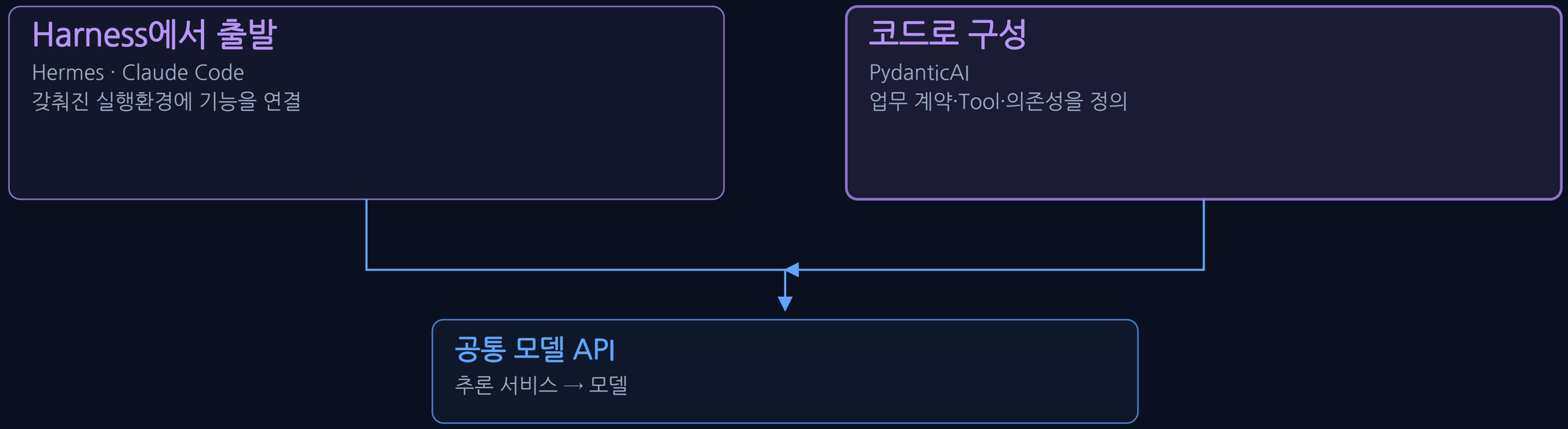
LangChain / LangGraph

Mistral AI Studio · vLLM / SGLang

서빙 엔진 내부와 실제 업무 적용은 다음 발표·3차로 연결한다.

완성된 실행환경과 개발용 구성요소

제품의 우열보다, 개발을 시작하는 위치와 남는 책임을 비교합니다.

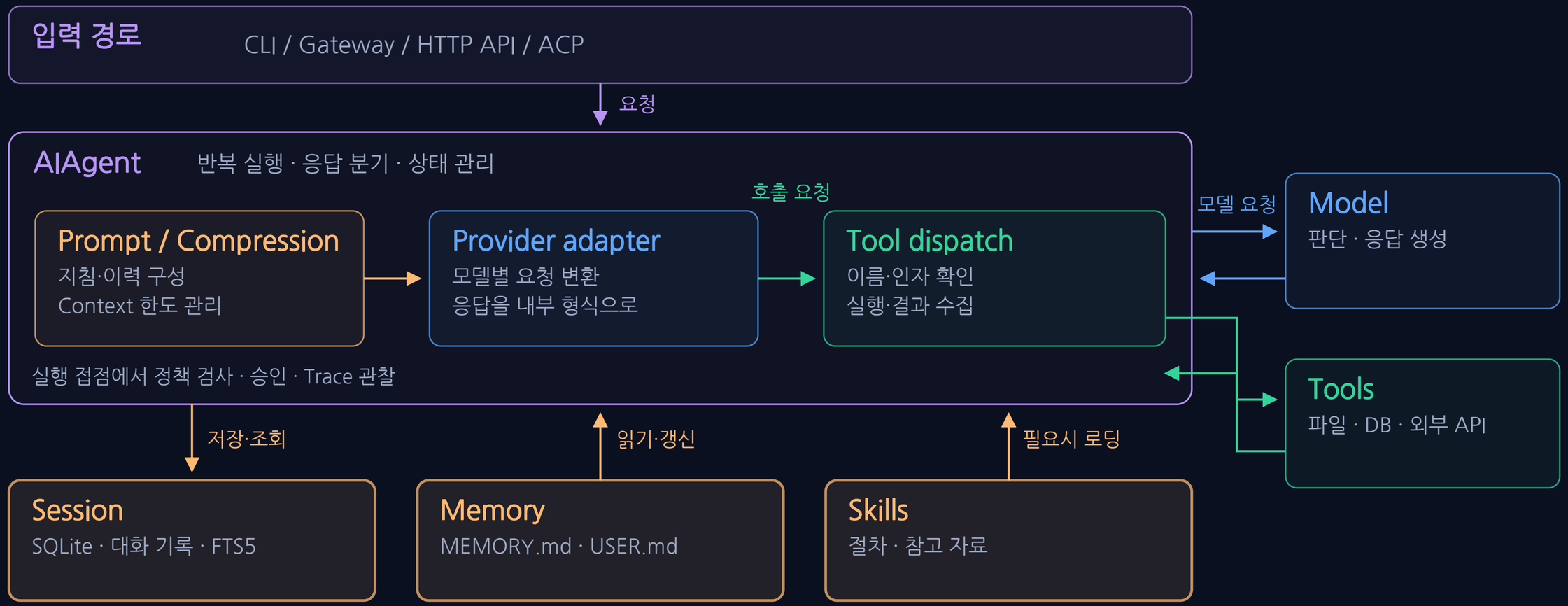


실행 루프는 양쪽 모두에 있다

> Harness / SDK는 배타적인 제품 계급이 아닌 개발 출발점이다.

Hermes는 반복 실행을 중심으로 기능을 묶습니다

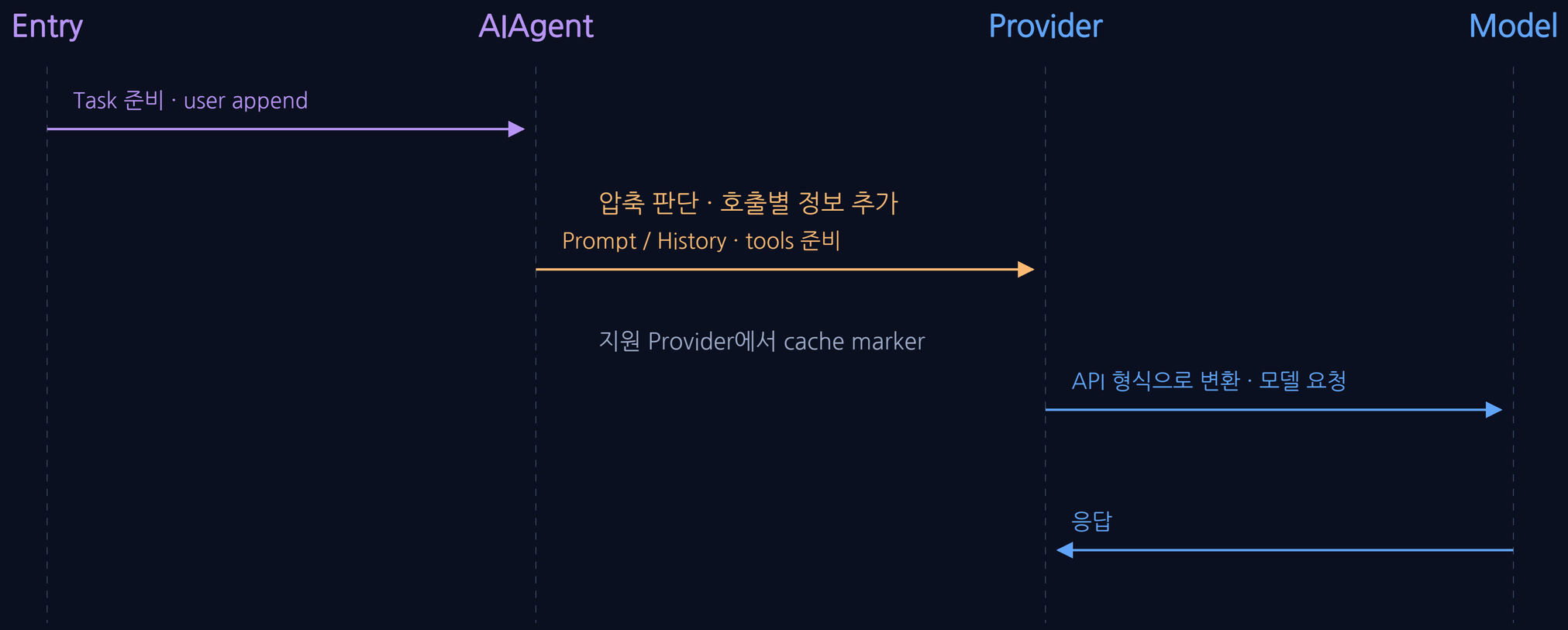
입력부터 모델·도구·저장소까지, 먼저 전체 지도를 봅니다.



모델 호출·도구 실행·상태 보존을 AIAgent가 연결한다.

한 Turn을 실제 요청으로 만드는 경로

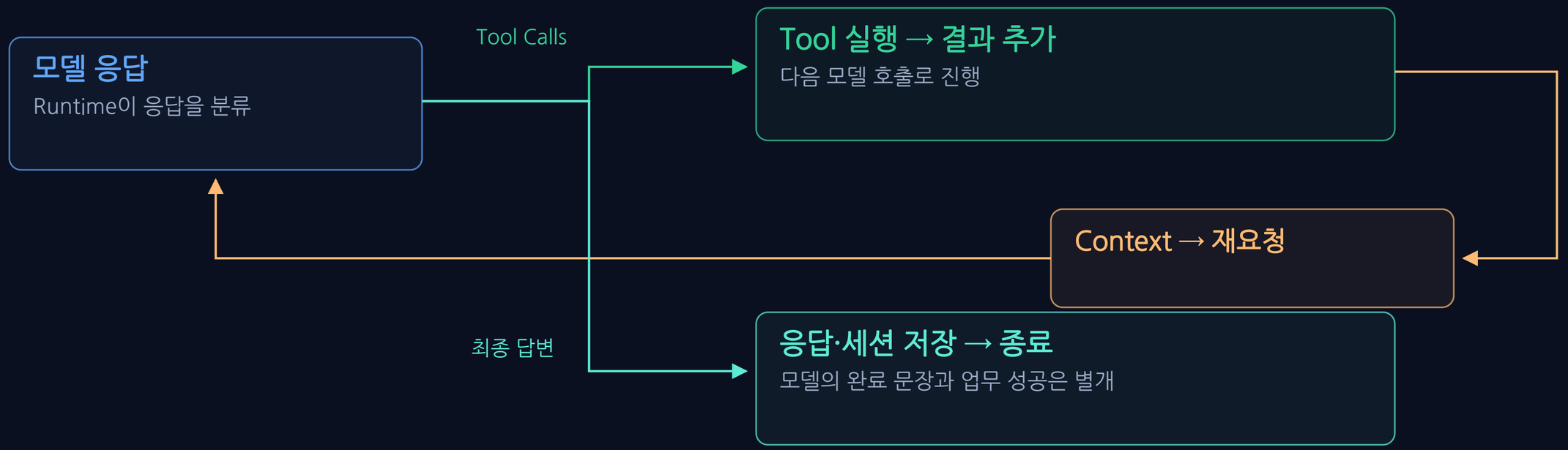
프레임워크 객체와 최종 API payload를 나눠서 봅니다.



API 직전 입력에는 지침·이력·도구 스키마·일시 정보가 함께 들어간다.

응답 분기가 루프와 종료를 결정합니다

Tool 오류와 모델 API 장애는 서로 다른 복구 경로입니다.



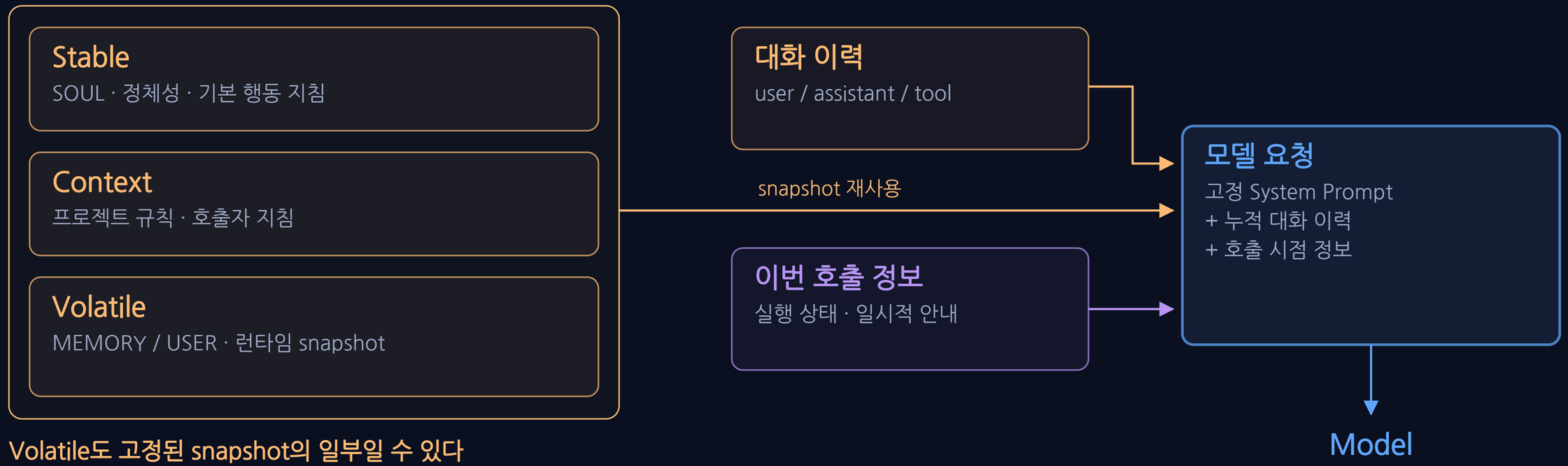
오류·중단·예산 Tool 오류 → 결과 메시지 / API 오류 → retry·fallback / 중단·예산 → 종료 경로

> 응답 종류·취소·실행 예산을 검사해 계속할지 멈출지 결정한다.

고정 Prompt와 호출별 정보는 따로 움직입니다

세션 snapshot에 대화 이력과 이번 호출 정보를 합쳐 요청을 구성합니다.

세션에서 재사용하는 System Prompt



Volatile도 고정된 snapshot의 일부일 수 있다

Provider adapter가 최종 API 형식으로 변환

> Volatile이라는 이름이 매 호출 재작성을 뜻하지는 않는다.

Prefix 재사용은 서빙 계층의 일입니다

같은 앞부분의 중간 계산을 재사용합니다. 입력 길이가 줄어드는 것은 아닙니다.



총 10,000 tokens · 설명용 수치



> Context 구성과 캐시 적중은 서로 다른 책임이다.

압축은 실행 이력의 정합성도 지켜야 합니다

단순히 오래된 텍스트를 자르는 작업으로 이해하면 안 됩니다.

Before

누적된 대화·Tool 결과

오래된 대화

이미 처리한 결정·관찰

Tool Call + Tool Result

대응 ID가 있는 실행 기록

최근 대화

현재 요청·미해결 항목

After

요약 + 최근 문맥 + Tool 쌍

요약과 보존 근거

중요 사실·결정·원본 경로

Call / Result 쌍 유지

한쪽만 남기는 축약은 피한다

최근 문맥 유지

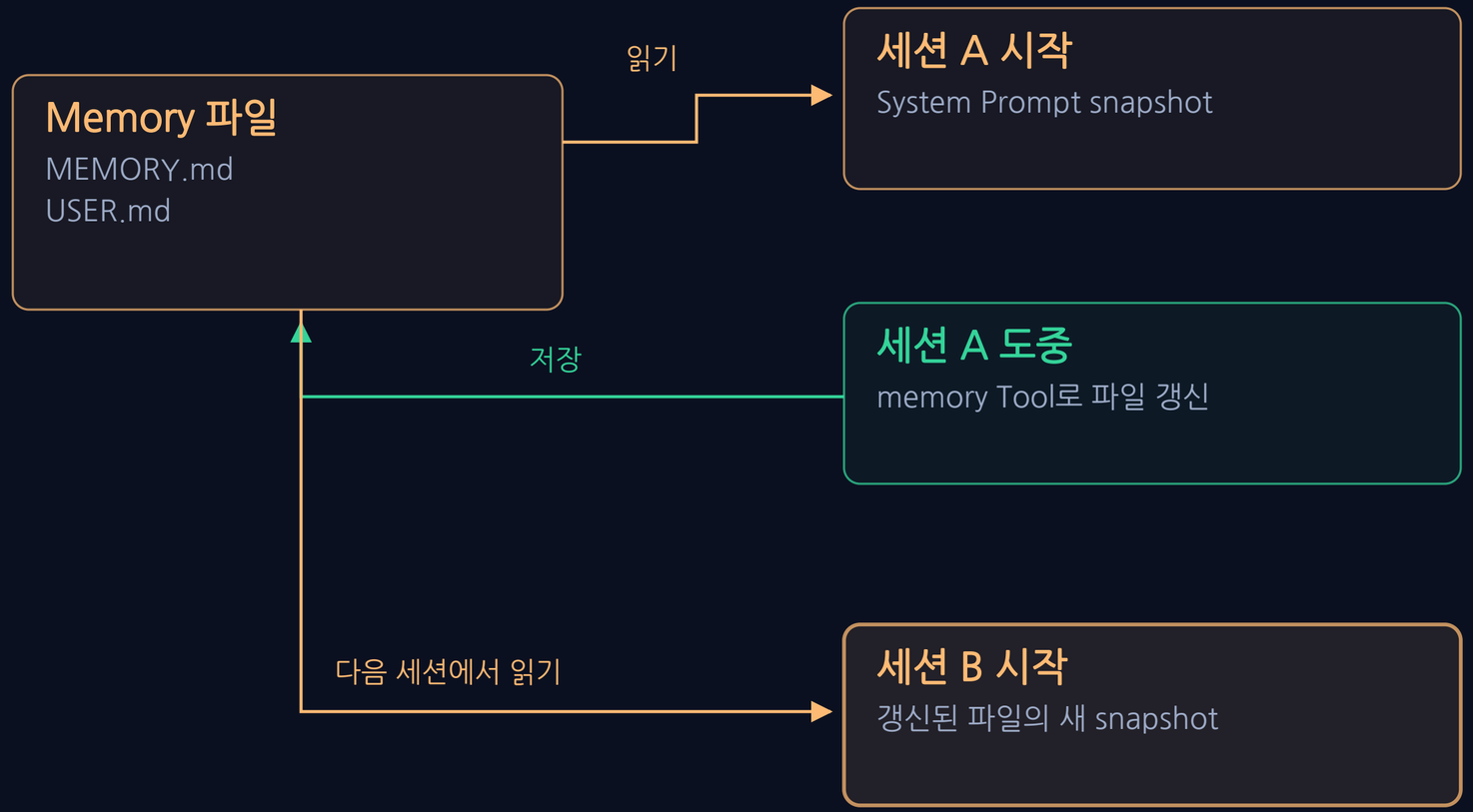
현재 작업을 이어갈 정보

발동 위치: Tool loop 안 / Gateway의 Turn 전 위생 처리

> 압축 임계값과 보존 범위는 실제 모델·설정 조건을 확인한다.

Memory 파일과 지금 보는 기억은 다릅니다

파일 저장·현재 세션의 snapshot·다음 세션 반영을 나눠서 봅니다.



현재 system snapshot은
자동 교체되지 않음

> 저장 성공이 현재 System Prompt의 즉시 교체를 뜻하지 않는다.

Session 검색은 전체 Memory의 대체물이 아닙니다

저장된 사실·과거 대화·외부 확장은 수명과 검색 방식이 다릅니다.

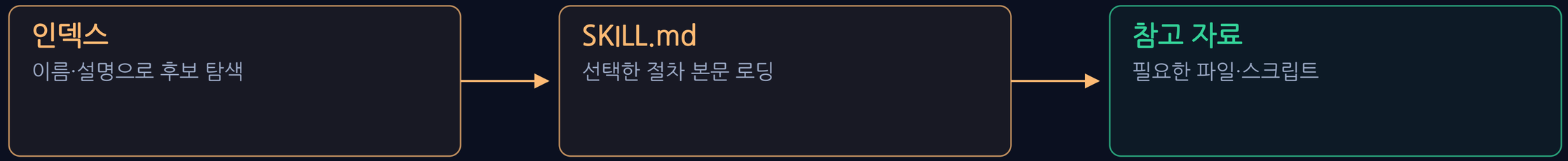
정보 계층	저장·접근	모델 입력으로 들어오는 방식
Built-in Memory	MEMORY.md / USER.md	세션 시작 snapshot
Session History	state.db · SQLite / FTS5	session_search로 선택한 기록
External Provider	별도 저장소·Plugin	확장 구현의 조회·주입

정확한 장비 ID 질의와 표현이 달라진 사례 질의를 나눠 평가한다.

> 전체 DB가 자동 주입되는 것이 아니라 필요한 기록을 선택한다.

Skill은 필요할 때 읽고 유지 관리합니다

실제 실행은 Tool이 맡고, Skill은 절차와 자료를 제공합니다.



문서형 절차 패키지

```
wafer-anomaly-review/
SKILL.md
references/
scripts/
```

유지 관리

skill_manage · 생성·수정·삭제

Curator

active → stale → archived

Agent 생성 Skill의 사용 이력 기반

> 생성된 절차가 재사용되므로 근거·검토·버전 관리가 필요하다.

도구 등록·외부 연결·확장 Hook은 다른 접점입니다

Schema 준비와 실제 실행을 연결하되, 확장 지점을 구별합니다.



memory / delegate_task 등 Agent 상태 도구에는 별도 처리 경로가 있을 수 있다.

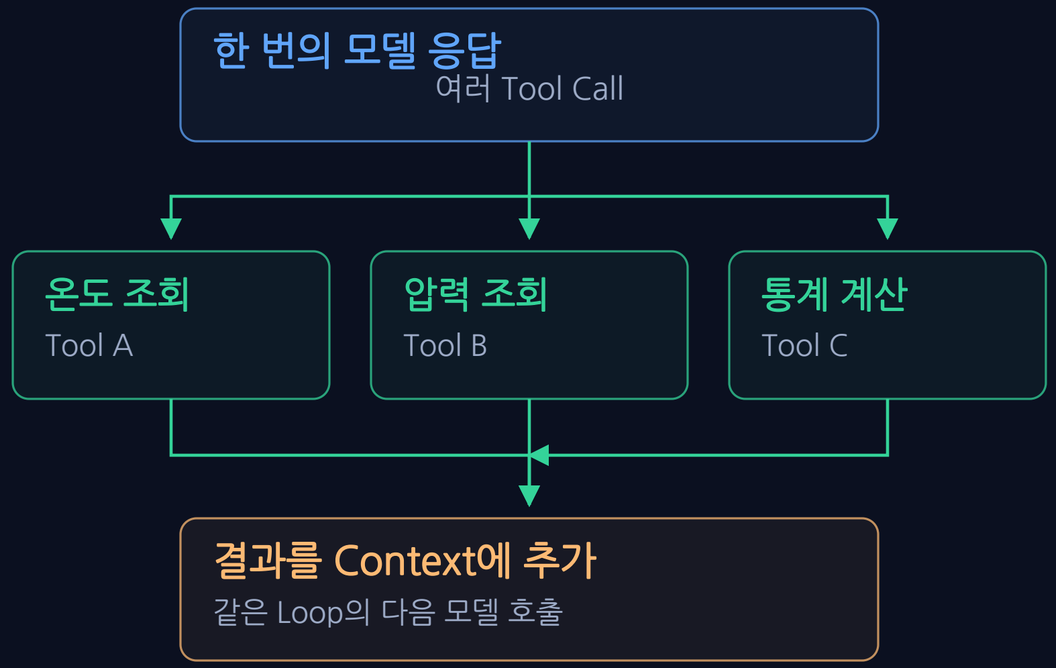
> 단일 중앙 Hook이 모든 내부 실행을 관찰한다고 단정하지 않는다.

병렬 Tool과 Child Agent는 실행 단위가 다릅니다

함수를 나란히 실행하는 것과 독립적인 Agent Loop를 위임하는 것은 다릅니다.

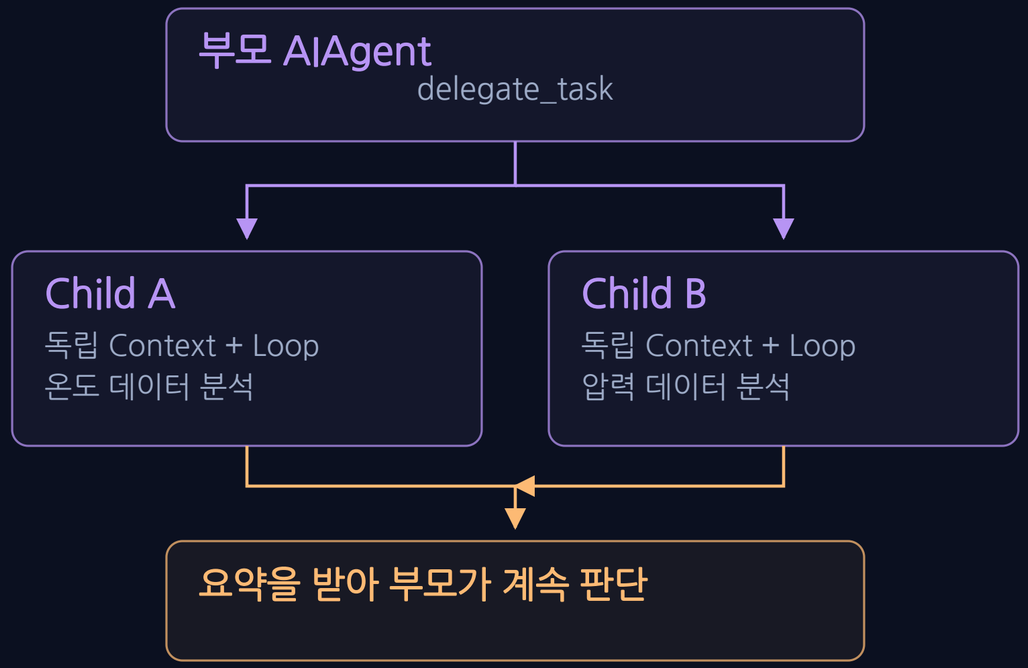
여러 Tool

한 Loop가 요청하고 결과를 모은다



여러 Child Agent

각자 모델 호출과 Context를 가진다

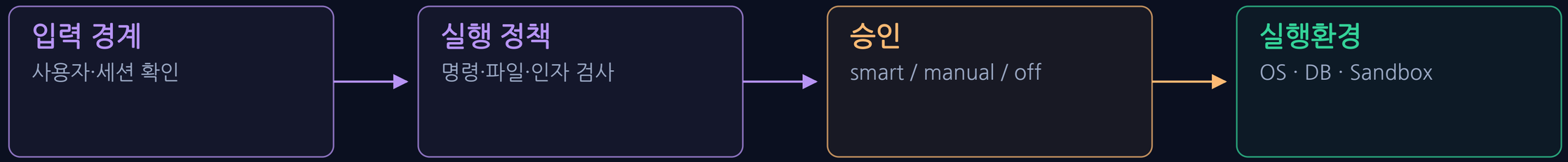


공유 파일·DB 권한은 별도로 격리해야 한다

> 자식의 결과는 부모로 돌아오며, 부모가 다음 판단을 이어간다.

정책 경계를 확인하고 Hermes Demo로 갑니다

행동의 허가와 실행 자원·복구는 서로 다른 통제 계층입니다.



패턴 탐지 + 실제 권한
위험 탐지와 접근 제어를 함께 사용
무인 실행의 deny 정책 별도

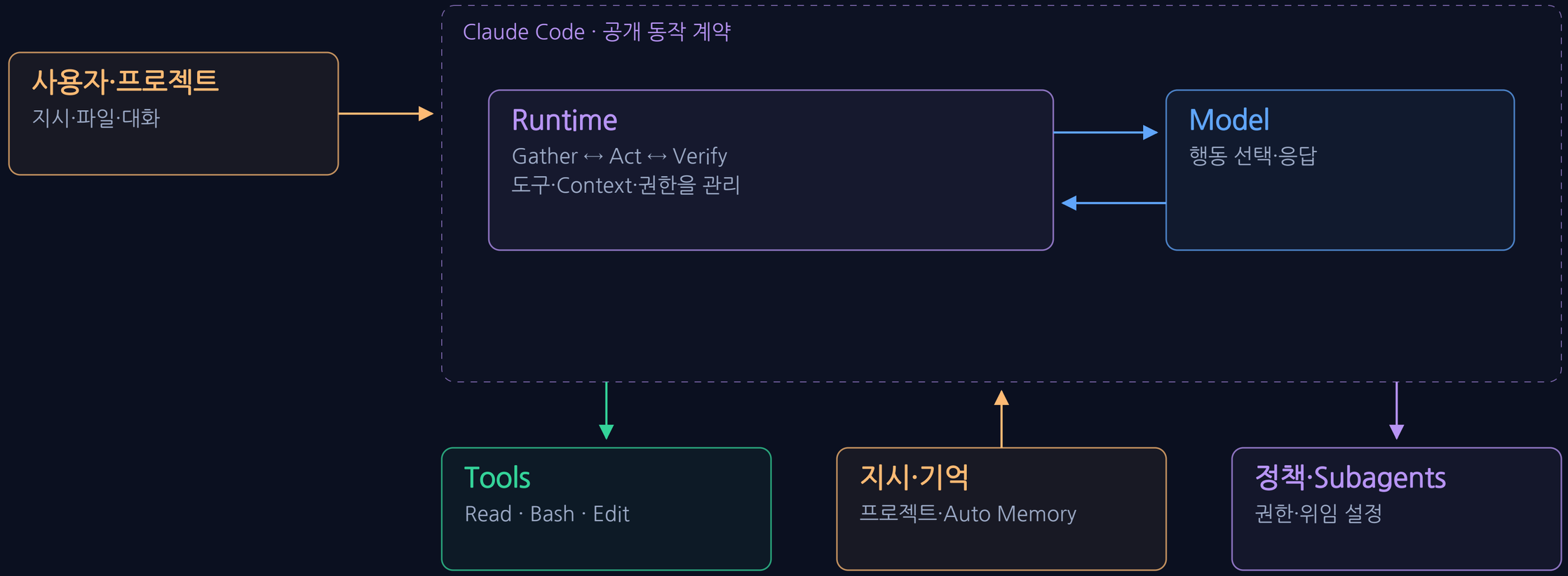
복구 Checkpoint
opt-in · 지원 범위에 따른 복구
외부 변경 전체 rollback은 아님

Demo 1 Context → Tool → 세션·기억 저장

> 실제 실행 기록에서 Context·Tool·저장 상태를 확인한다.

Claude Code도 Harness에서 출발합니다

맥락 수집·행동·검증이 섞여 진행되는 실행환경입니다.



공개 인터페이스를 비교하며, 비공개 엔진의 내부 구현을 창작하지 않는다.

프로젝트 지시와 Auto Memory를 분리합니다

파일 이름이 같아도 제품별 저장·로드 정책은 다릅니다.

제품	사람이 작성하는 지시	Agent가 관리하는 기억
Hermes	SOUL · 프로젝트 지침	MEMORY.md / USER.md 세션 snapshot
Claude Code	CLAUDE.md 필요시 @AGENTS.md import	Auto Memory 인덱스 필요시 세부 파일 읽기

지침 파일은 행동을 안내하고, 실제 접근 권한은 실행 계층에서 집행한다.

> 지시와 기억은 실제 파일·DB·네트워크 접근 권한을 대신하지 않는다.

Subagent 반환과 Team 협업은 다릅니다

누가 누구에게 결과를 전달하고, 다음 행동을 결정하는지 봅니다.

Subagent

부모의 위임을 수행하고 결과를 반환



결과는 부모에게 보고

Agent Teams

독립 세션·공유 작업·직접 메시지



공유 작업 목록 · 실험적 제약 · 조정 비용

> Context 분리와 실제 자원 격리·협업 충돌 관리는 별개다.

Slash command는 실행 상태를 조작하는 입구입니다

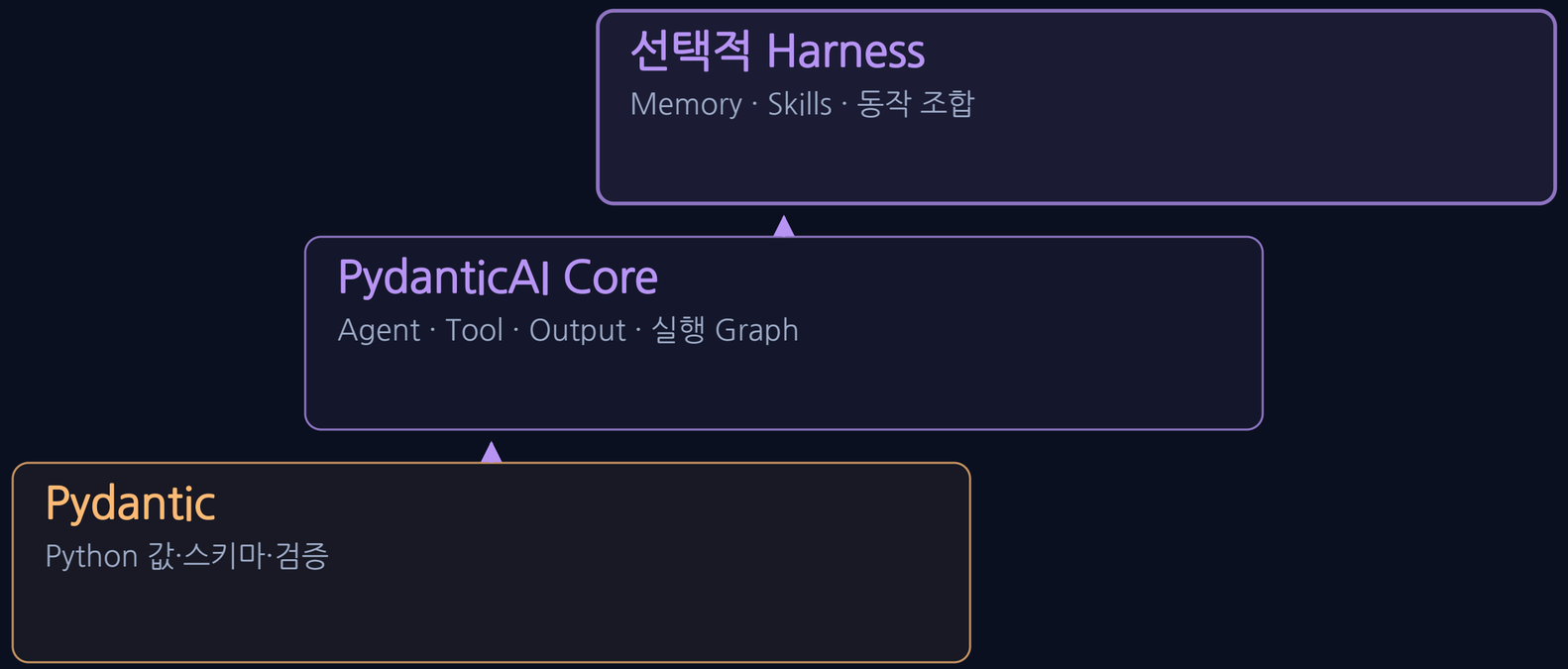
명령을 실행하지 않고, 그 뒤에 있는 상태 관리 기능을 비교합니다.

제품·입구	명령	바뀌는 상태
Hermes CLI	hermes sessions refresh-prompts --session SESSION_ID	저장된 Prompt 갱신 경로
Hermes Slash	/memory pending	보류된 기억 쓰기 검토
Claude Code	/resume · /compact	대화 선택 · Context 압축

재개·Prompt 갱신·Memory 검토를 같은 동작으로 합치지 않는다.

Pydantic → PydanticAI → 선택적 Harness

값의 검증에서 실행 엔진으로, 필요한 기능을 조합하는 구조로 확장합니다.



함께 연결하는 도구

pydantic-graph

명시적 Graph·Workflow

Evals / Logfire

평가와 실행 관찰

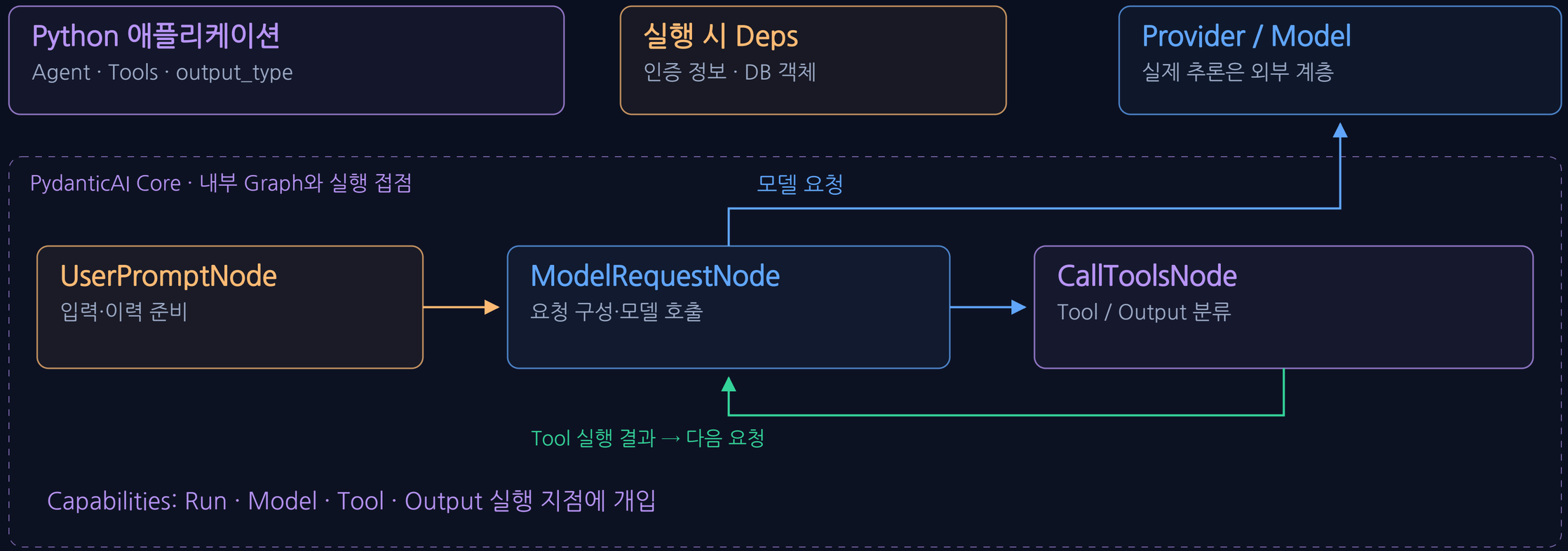
Provider / Model

모델 추론은 외부 계층

> Core만으로 Agent를 만들 수 있고 Harness는 선택적으로 추가한다.

PydanticAI 전체 아키텍처

개발자 정의와 SDK가 실행하는 Graph를 분리해서 봅니다.



연결: History 저장소 / Tool 서비스 / Output 검증 / 선택 Harness

> 내부 Graph는 pydantic-graph이며, 업무 데이터·권한·저장은 앱과 연결한다.

개발자는 업무 계약과 Tool을 작성합니다

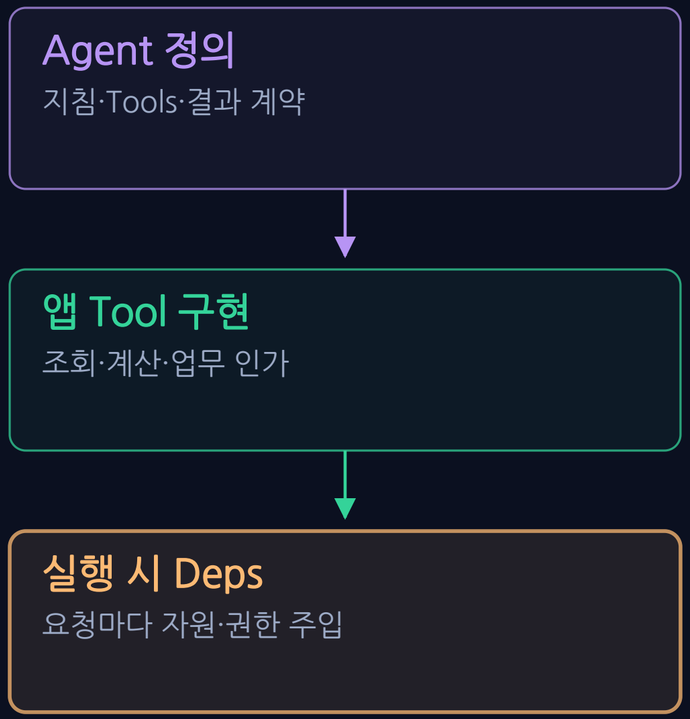
SDK의 루프를 다시 만들지 않고, 업무별 정의와 실행 함수를 연결합니다.

실제 API 축약 · 앱 정의/연결 설정은 노트 참고

```
agent = Agent(model, deps_type=DemoDeps,
instructions='도구 근거로만 답하라.')

@agent.tool
async def get_bin_summary(
ctx: RunContext[DemoDeps], lot_id: str
) -> dict:
return await ctx.deps.query(lot_id)

result = await agent.run(
'LOT-DEMO의 BIN-X를 조회해줘', deps=deps)
```

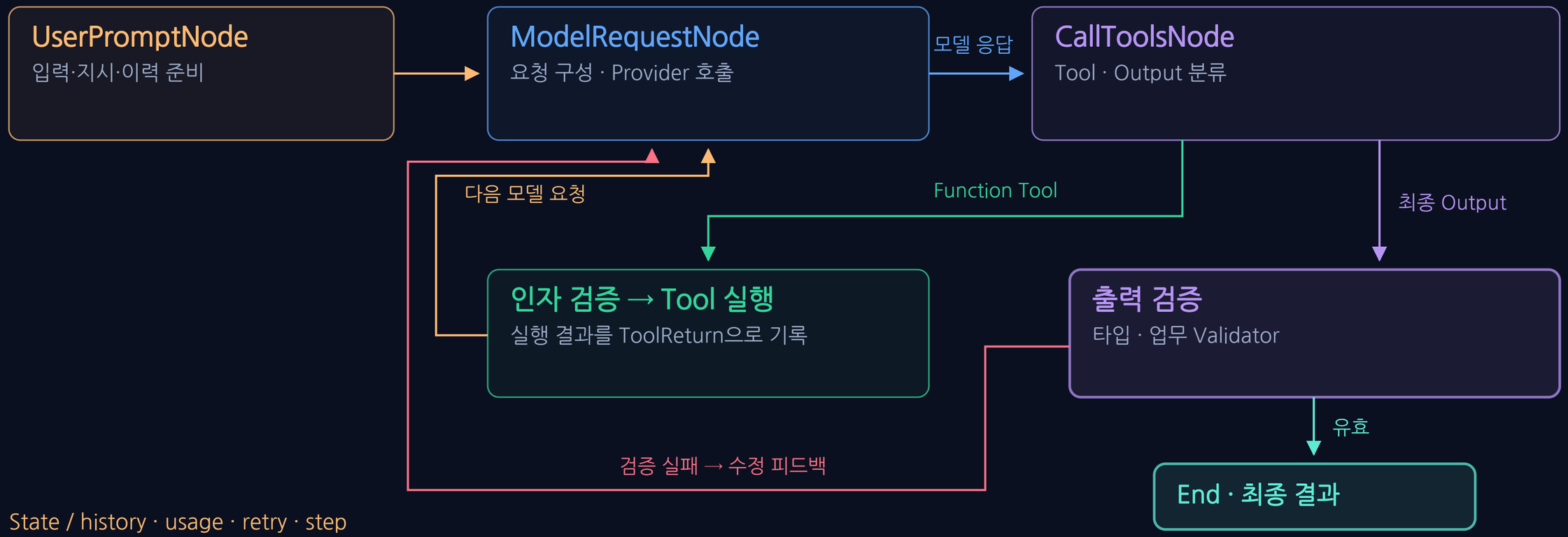


query와 DemoDeps는 앱 코드이며, 실제 인가는 Tool·서비스에서 검사한다.

세 Graph 노드가 다음 실행을 정합니다

모델 응답은 Tool 실행·최종 출력·수정 요청의 서로 다른 경로로 이어집니다.

PydanticAI 내부 실행 Graph · pydantic-graph

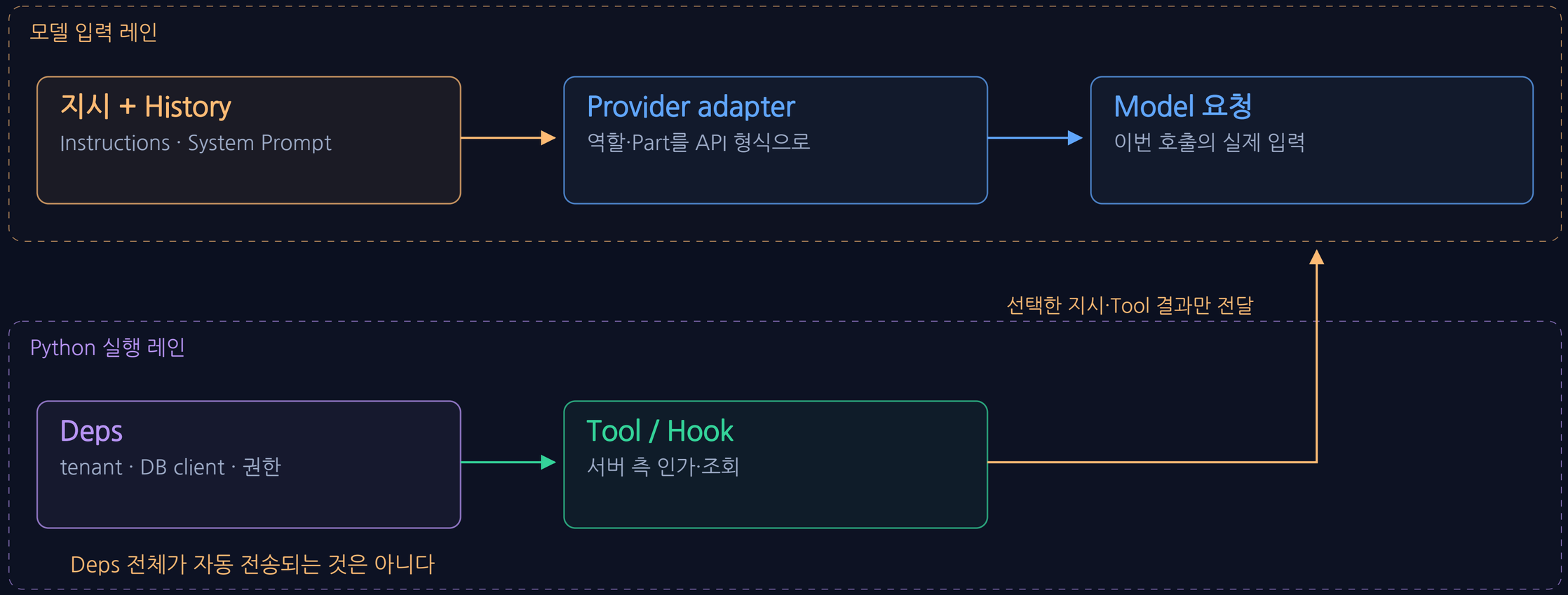


State / history · usage · retry · step

> 유효한 Output은 종료로, Tool 결과와 수정 피드백은 다음 요청으로 간다.

지시·History·Deps는 같은 입력이 아닙니다

Python 자원과 모델에 전달하는 정보 사이의 경계를 명시합니다.



인증 객체·DB 연결은 deps로 주입하되, Tool 결과의 정보 유출도 따로 검사한다.

Tool은 스키마 준비·검증·실행을 거칩니다

모델에 보이는 함수 계약과 Python에서 실행하는 함수는 연결되지만 다릅니다.



등록 방식	Python 실행 문맥	사용 예
@agent.tool	RunContext를 받음	tenant·DB·권한이 필요한 조회
@agent.tool_plain	일반 인자	독립 계산·변환

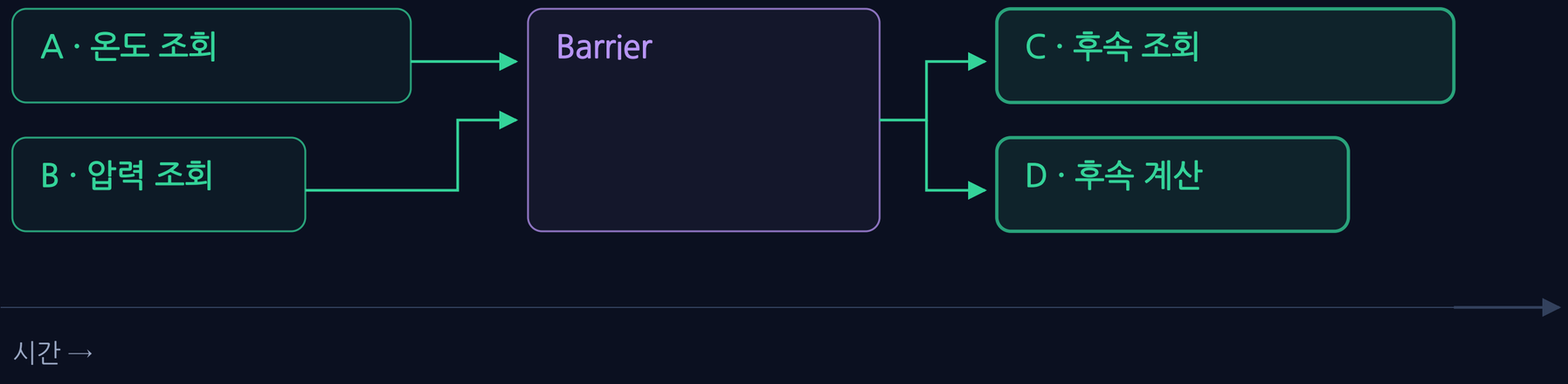
> Toolset 노출 필터와 실제 실행 인가는 서로 다른 단계다.

병렬 실행은 의존성 없는 구간을 겹칩니다

앞 구간 완료 → barrier 단독 실행 → 뒤 구간 시작의 관계를 봅니다.

모델이 제안한 순서 A · B → sequential Tool → C · D

실제 실행



async Tool: task 동시 스케줄

sync Tool: worker thread

sequential=True는 실행 장벽이며 데이터 snapshot·트랜잭션을 만들지는 않는다.

> 병렬 조회도 같은 분석 대상·시점·snapshot 계약이 필요하다.

출력 형식과 결과의 정확성은 따로 검증합니다

Schema를 통과했다고 계산·인과·예측이 옳다는 뜻은 아닙니다.

ToolOutput

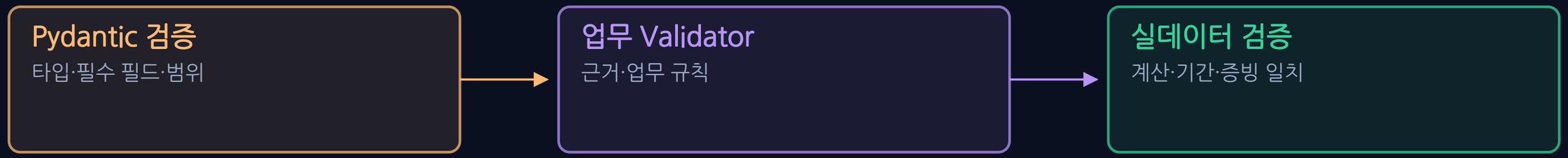
출력용 Tool Schema 제공
모델이 구조화 호출 생성

NativeOutput

Provider의 구조화 기능
모델·스키마 지원 확인

PromptedOutput

형식 지시 + 파싱·검증
형식 위반은 수정 경로



output_type은 최종 결과 계약 · 개별 Tool return type과 목적이 다르다

> 형식 통과와 도메인 정확성을 서로 다른 검증 기준으로 평가한다.

EndStrategy는 실제 실행 범위를 바꿉니다

같은 모델 응답이라도 실행되는 Tool과 최종화 시점이 달라집니다.

모델이 한 응답에서 제안한 순서

조회 A (Function Tool) → 출력 O1 (Output Tool) → 게시 Mock B (Function Tool) → 출력 O2 (Output Tool)

전제: O1·O2 모두 유효 / Retry 없음 / 비스트리밍 / barrier 없음

	조회 A (Function Tool)	출력 O1 (Output Tool)	게시 Mock B (Function Tool)	출력 O2 (Output Tool)
early O1만 처리하고 종료	생략	O1 선택	생략	생략
graceful 뒤 Function도 실행	실행	O1 선택	실행	생략
exhaustive 모두 실행 · 기본 병렬	실행	O1 선택	실행	실행 · 미선택

최종 결과는 emission 순서의 첫 유효 Output · 완료가 빠른 순서가 아니다

> graceful에서도 최종 Output 뒤 Function Tool이 실행될 수 있다.

Retry를 발생 위치별로 나눕니다

모델에게 수정시킬 문제와 통신 장애·업무 실패를 구별합니다.

발생 위치	되돌리는 방식	별도 설계
인자·출력 오류	검증 실패 → 모델 수정 피드백	ModelRetry · 재시도 한도
Provider/API 장애	429·5xx·연결 실패 → 통신 복구	백오프 · timeout · 중첩 retry
업무 판단 실패	유효한 형식이어도 분석은 틀릴 수 있음	정답·평가·사람 검토

UsageLimits + 시간 제한 + Tool 부작용 중복 방지는 별도 관리

> SDK·HTTP·프록시 retry가 겹치면 실제 wire 요청 수가 증폭될 수 있다.

Hook을 기능 단위 Capability로 합성합니다

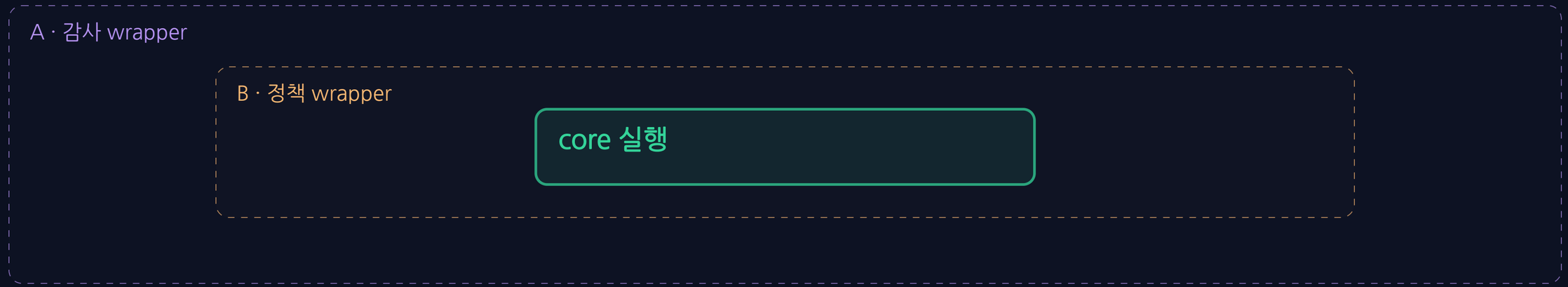
재사용 단위와 실행 순서, Run별 상태 수명을 함께 설계합니다.

Hook

모델 요청·Tool·Run의 실행 지점

Capability

지시·도구·Hook·설정을 묶는 기능



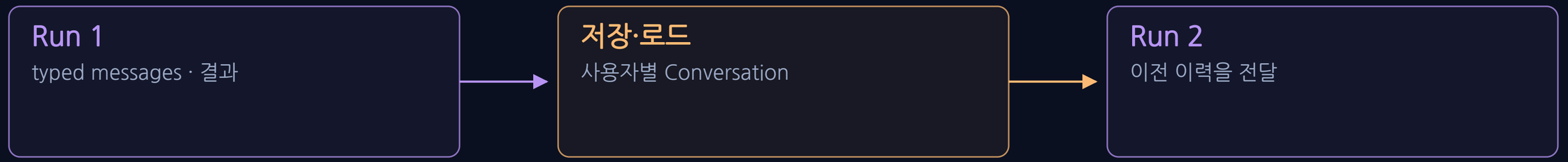
A.enter → B.enter → core → B.exit → A.exit

handler를 생략하는 wrapper는 안쪽 정책 Hook도 건너뛸 수 있다.

> 정책이 필요한 실행 접점과 wrapper의 우회 가능성을 검증한다.

여러 Run의 대화는 명시적으로 이어 붙입니다

Agent 객체를 재사용하는 것과 사용자 대화를 복원하는 것은 다릅니다.



실제 API 축약 · 앱 정의/연결 설정은 노트 참고

```
first = await agent.run('BIN-X를 조회해줘')
second = await agent.run(
'분모는?', message_history=first.all_messages())
```

함께 이어갈 상태

history · usage · ID
현행 conversation API

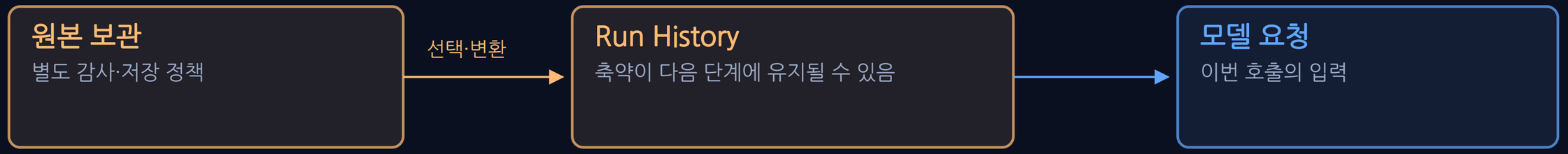
앱이 인증·저장·동시 갱신·TTL과 대화 누적 예산을 관리한다.

압축 전략은 제공되지만 보존 정책은 설계합니다

모두 직접 구현할 필요는 없지만, 요청 축약과 원본 저장은 구별해야 합니다.

Core
Provider-native compaction
지원 모델·API 조건 필요

별도 Harness
clear · sliding window · summarize
모델 비종속 전략 선택

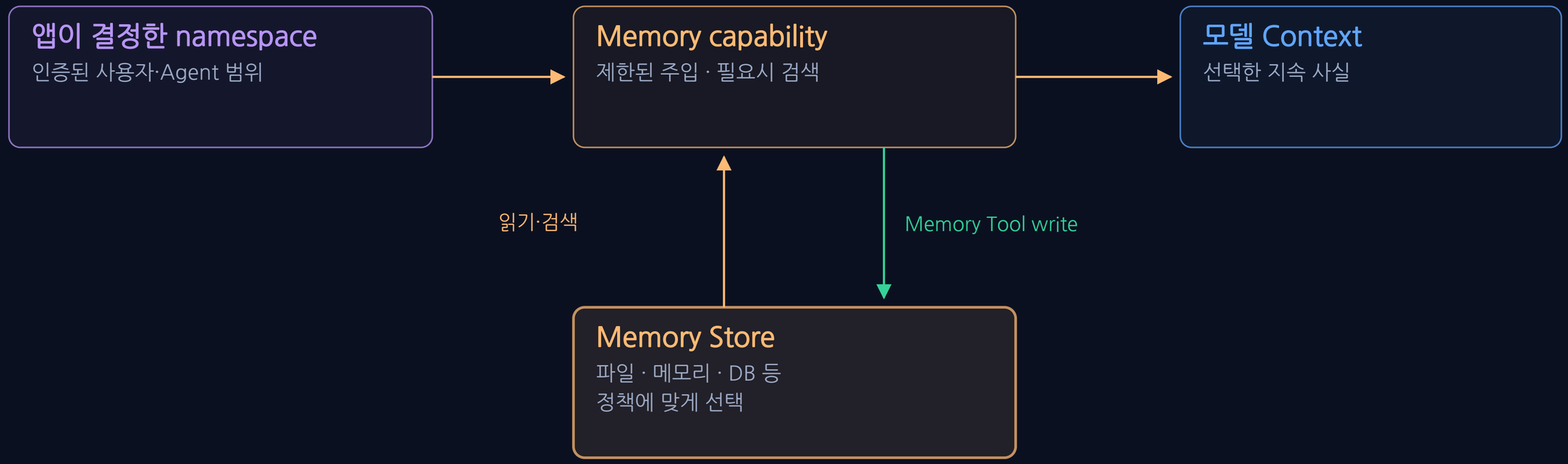


Tool 쌍·정확 수치·근거 경로를 보존하고 원본 복원을 자동 보장하지 않는다.

> vLLM 이관 기본 경로는 Provider 전용 compaction에 의존하지 않는다.

Harness Memory는 저장·주입·검색을 묶습니다

대화 이력과 다른 수명의 정보를 다루는 선택 확장입니다.



namespace는 저장 범위이며, 인증·DB 접근 권한을 대신하지 않는다.

> 무엇을 기억하고 누가 읽고 갱신할지, SOP 만료 정책까지 정한다.

Skill 로딩과 MCP 호출을 따로 연결합니다

기본 추상화가 제공되어도 실제 절차·서버·인증은 구성해야 합니다.

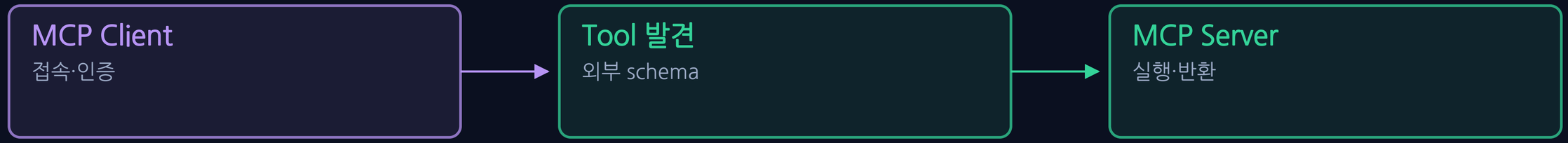
Skills

문서형 절차를 모델 입력에 제공



MCP

외부 기능의 발견·호출을 연결



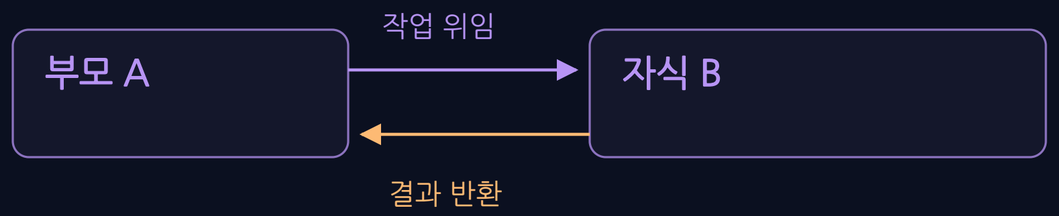
> Skill은 입력 경로, MCP는 외부 Tool 실행 연결 경로다.

위임·라우팅·Graph·Advisor의 제어권

다음 행동 주체와 반환 위치, 공유 상태를 구분합니다.

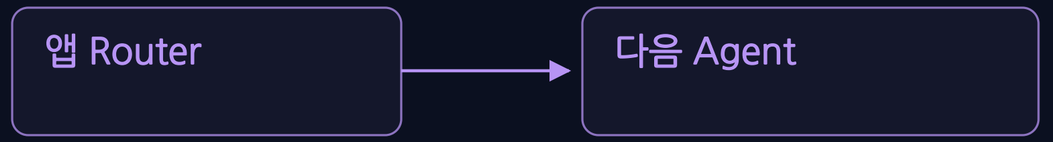
도구 내부 위임

부모 → tool(B.run) → 부모



Programmatic Handoff

앱 코드가 다음 Agent를 선택



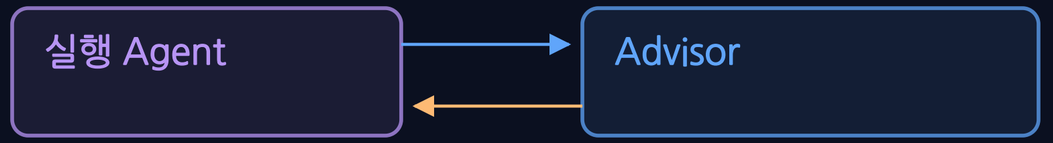
Graph Workflow

노드·상태·분기를 명시



Harness Subagents / Advisor

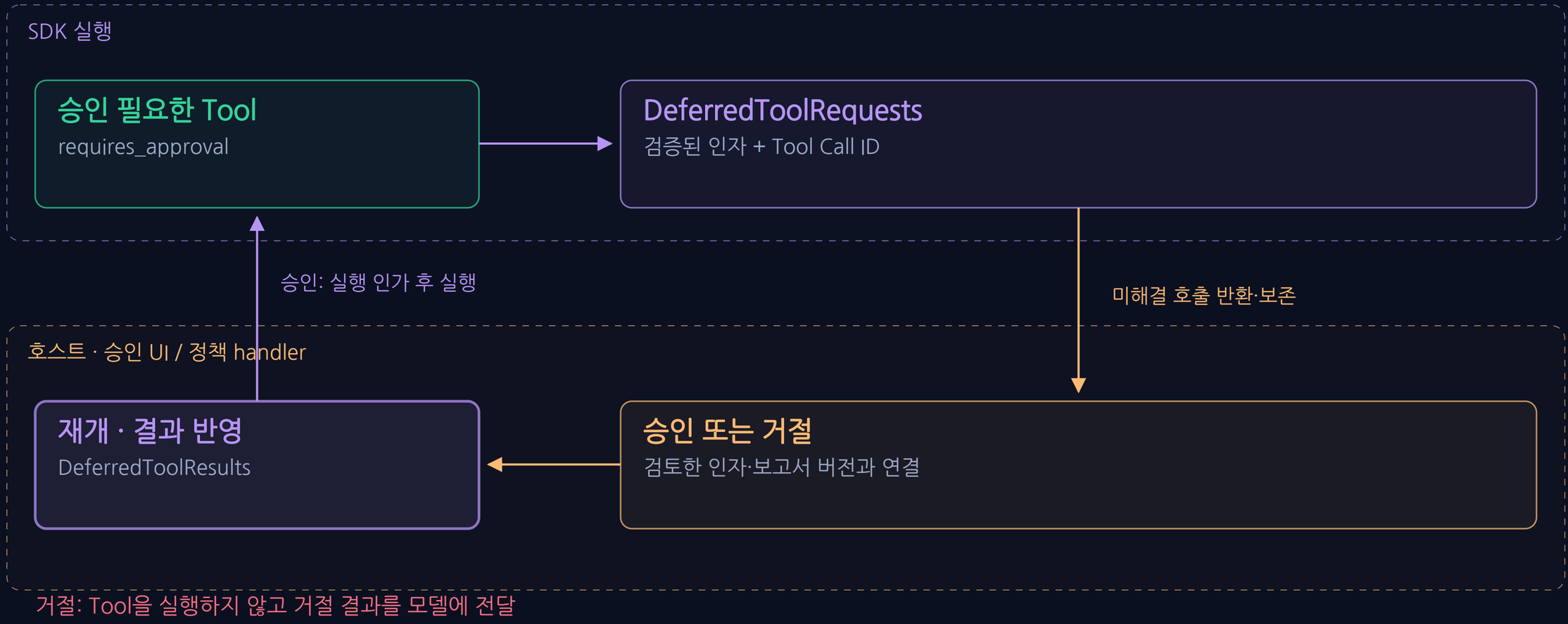
하위 작업과 조언을 역할별로 구성



> delegation은 부모 반환 패턴이며 다른 SDK의 native handoff API와 동일하지 않다.

승인은 보류 Call을 해결하고 재개하는 계약입니다

승인 플래그만으로 업무 UI나 인증·영속 복구가 완성되지는 않습니다.



Call ID·검토한 인자·데이터 버전을 묶고 실행 시 권한을 다시 검사한다.

Core에 Harness를 더해도 업무 책임은 남습니다

제공받는 실행 접점과 직접 설계할 업무 계약을 함께 봅니다.

책임	Core	선택 Harness	우리 애플리케이션
실행	Loop · Tools · Output	동작 스택·위임	업무 함수·종료 기준
정보	메시지·이력 API	Memory · Skills	데이터 계약·보존·삭제
통제	검증·Hook·승인 경로	정책 조합	인증·인가·감사·운영

Logfire / OTel은 실행 관찰, Evals는 목표 기준 평가에 연결한다.

> 이점은 타입 문법을 넘어 검증·실행·최종화·재개를 같은 Runtime에서 조합하는 데 있다.

코드와 내부 전이를 PydanticAI Demo에서 잇습니다

최종화 정책 시험은 live 응답과 구별된 고정 응답 시험으로 관찰합니다.

개발자 코드

```
Agent(...)

@agent.tool
...

agent.run(...)
```



Demo 2 Core 먼저 → 선택 Harness의 변화 관찰

> 실제 코드 정의가 Graph·최종 입력·Tool 결과로 이어지는 경로를 본다.

중앙 실행과 로컬 실행은 책임 배치가 다릅니다

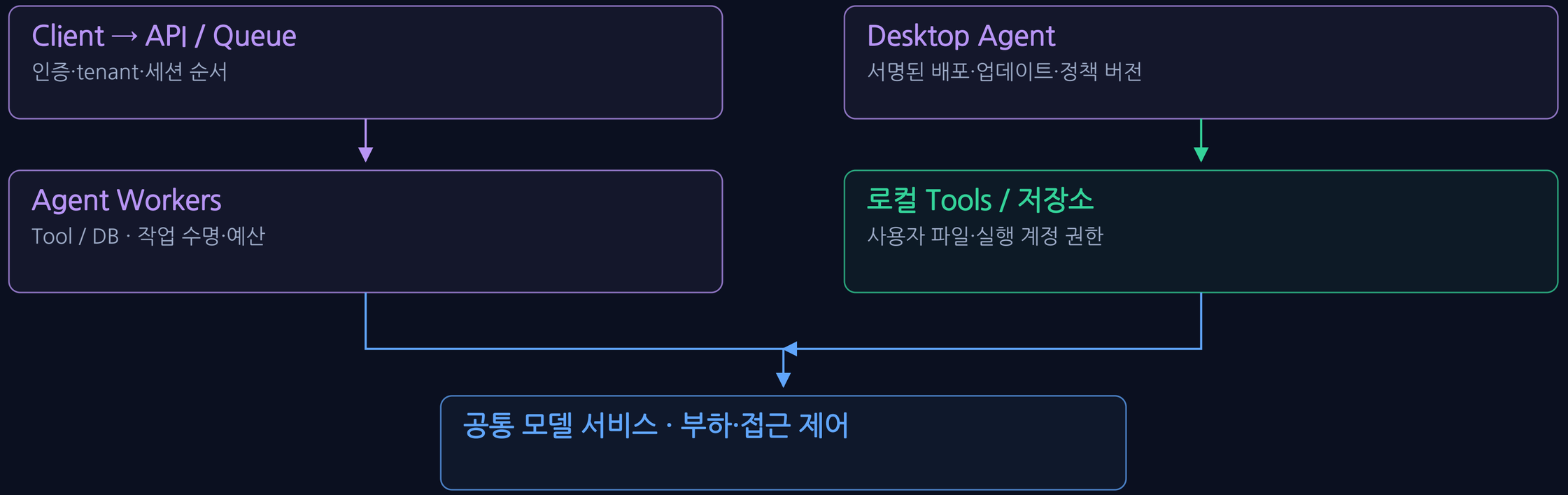
Agent 실행 자원과 모델 서빙 자원을 분리해서 측정합니다.

중앙 Agent 실행

공통 운영 + 사용자·작업 격리

사용자 로컬 실행

설치·업데이트 + 로컬 실행 권한



> 로컬 Agent여도 모델 요청으로 데이터가 나가므로 데이터 경로를 검토한다.

Application

분석에 필요한 검증 기준을 정합니다

실행 증거와 계산·근거·정책 충족 여부를 연결합니다.

Trace · 무엇이 실행되었나

관찰 가능한 실행 증거

실행 기록

Run / Request / Call ID
실제 인자·Tool 결과
승인·재개·오류 경로
사용한 데이터 snapshot

Evaluation · 옳게 수행했나

업무 목표와 제약 기준

업무 검증

분자·분모·기간 일치
근거·SOP revision 확인
승인 인자와 실행 일치
예측 시간 분할·누수 확인

같은 Tool을 호출했다는 사실만으로 같은 근거·같은 결과를 보장하지 않는다.

> 조회·관측·원인 가설·예측을 구분하고 실측 없이 성능을 주장하지 않는다.

제공받는 실행과 직접 만드는 계약

프레임워크 선택은 실행 책임을 나누는 결정입니다.

01 **Hermes**

상태·기억·정책이 실제로 어떻게 움직이는가

02 **PydanticAI**

검증·실행·최종화·확장을 어떻게 조합하는가

03 **우리의 계약**

데이터·권한·업무 검증을 어떻게 보장하는가

> 다음 발표와 3차에서는 실제 Agent를 만들고 이 계약으로 검증한다.